CS 218 - Design & Analysis of Algorithms

End-Semester Exam

Time: 3 hrs. | Total Marks - 50

Instructions. Please be concise, clear, and technically accurate. Describe all algorithms using pseudocode, followed by a brief discussion. Wherever applicable, draw upon the algorithmic techniques covered in class, and ensure that proofs are rigorous; lack of formality may result in mark deductions. The bonus question will be extra marks and will be added to the total end semester marks.

Question 1 (6 Marks)**.** Suppose a problem X is NP-complete. For each of the following statements, write TRUE or FALSE. Each part carries 1.5 marks. A brief explanation is required for each of the following questions.

(a) We know for sure that X does not admit a polynomial-time algorithm.

(b) We know for sure that X admits an exponential-time algorithm of the form $\mathcal{O}(2^{n^c})$, where n denotes the input size and c is a constant.

(c) The complement of X is also in NP. (A problem Y is called the complement of X if, for every input α, a is a yes-instance of X if and only if a is a no-instance of Y.)

(d) Designing a polynomial-time algorithm for X would imply that P=NP.

Solution:

(a) FALSE. We do *not* know for sure that X lacks a polynomial-time algorithm. That would require proving P $\neq$ NP, which remains an open problem. If P = NP, then every NP-complete problem — including X — would have a poly-time algorithm.

(b) TRUE. Since X is in NP, there is a polynomial-time verifier for X. For any input of size n, the certificate has polynomial length, say n^c. We enumerate all 2^{n^c} candidate bit-strings of that length and verify each — giving an $\mathcal{O}(2^{n^c})$ algorithm. Such an algorithm always exists for any NP problem.

(c) FALSE. The complement of X is in co-NP. Whether co-NP = NP is unknown. The complement is in NP only if NP is closed under complementation (i.e., NP = co-NP), which has not been proven. So we cannot assert the complement is in NP.

(d) TRUE. By NP-completeness, every problem in NP reduces to X in polynomial time. A poly-time algorithm for X would then yield poly-time algorithms for all NP problems, proving P = NP.

Rubric: 0.5 marks for correctly identifying True/False; 1 mark for a correct and complete explanation; 0.5 marks for a partially correct explanation; 0 marks for an incorrect or missing explanation.

Question 2 (8 Marks)**.** Suppose there exists a very fast algorithm for the INDEPENDENT SET problem: given a graph $G = (V, E)$ and an integer k, it decides whether G contains an independent set of size k. A set of vertices is called independent if no two vertices in the set are adjacent. We are given a list of n courses offered in the next semester and a set of available time slots. For each course i, the instructor specifies a subset $L_i \subseteq \{1, 2, ..., l\}$ of acceptable time slots. We must assign each course i to exactly one slot from L_i. Additionally, we are given a list of pairs of courses (i, j) such that courses i and j must not be scheduled in the same time slot.

This problem is NP-hard, so we cannot hope for a polynomial-time algorithm in general. However, suppose we are given access to the above fast program for the INDEPENDENT SET problem. How can we use this program to solve the course slot-allocation problem? Hint: For every course i, create $|L_i|$ vertices, one for each slot in L_i.

[Marks distribution: 6 marks for the reduction, 2 marks for the correctness argument.]

Solution:

Construction:
Build a graph $G = (V, E)$ as follows:

1. For each course i and each acceptable slot $s \in L_i$, create a vertex $v_{i,s}$. There are at most $n \cdot \ell$ vertices total.
2. **Same-course edges:** For each course i and two distinct slots $s \neq s' \in L_i$, add edge $(v_{i,s}, v_{i,s'})$. This ensures at most one slot is chosen per course.
3. **Conflict edges:** For each conflict pair (i, j) and each slot $s \in L_i \cap L_j$, add edge $(v_{i,s}, v_{j,s})$. This prevents conflicting courses from sharing a slot.

Query the Independent Set oracle with graph G and $k = n$.

Correctness Argument:
(⇒) Given a valid schedule assigning course i to slot $\sigma(i) \in L_i$, the n vertices $\{v_{i,\sigma(i)}\}$ form an independent set of size n. No same-course edges are present (we picked one slot per course). No conflict edges are present (conflicting courses have different slots, so $v_{i,\sigma(i)}$ and $v_{j,\sigma(j)}$ are only adjacent if $\sigma(i) = \sigma(j)$, which the schedule forbids).

(⇐) Given an independent set S of size n, it contains exactly one vertex per course (otherwise some course has 0 or ≥ 2 vertices in S; ≥ 2 would create a same-course edge contradiction). Let $\sigma(i)$ be the slot of the unique vertex for course i. Since no conflict edge $(v_{i,s}, v_{j,s})$ is in S, no two conflicting courses share a slot. The assignment σ is valid.

Rubric: 2 marks for Same-course edges, 2 marks for conflict edges, 2 marks for the IND-SET oracle query, 1 mark for each side of the correctness argument

Question 3 (8 Marks)**.** Suppose you are given a directed graph $G = (V, E)$ where each edge $e \in E$ has a positive integer capacity c_e. Lets $s \in V$ be the source and $t \in V$ be the sink in G. You are also given a maximum s-t flow in G, defined by a flow value f_e on each edge e. The flow f is acyclic: there is no cycle in G on which all edges carry positive flow. The flow f is also integer-valued. Now suppose we pick a specific edge $e^* \in E$ and reduce its capacity by 1 unit. Show how to find a maximum flow in the resulting capacitated graph in time $\mathcal{O}(m + n)$ where m is the number of edges in G and n is the number of nodes.

Marks distribution: 4 marks for the algorithm, 3 marks for correctness, and 1 marks for the running time analysis.

Solution:

Algorithm:

Let the reduced edge be $e^* = (u, v)$.

```
Case 1: f(e*) <= c(e*) - 1
  The existing flow f is still feasible. Return f unchanged.

Case 2: f(e*) = c(e*)
  Step 1. Find a path P1 from s to u in the flow graph (edges with f(e)>0) via BFS.
  Step 2. Find a path P2 from v to t in the flow graph via BFS.
  Step 3. Reduce flow by 1 along P1 -> e* -> P2. (Flow becomes |f| - 1).
  Step 4. Build the residual graph G_f' and search for an augmenting s-t path.
  Step 5. If augmenting path found: augment by 1 unit. New max flow = |f|.
          Else: max flow = |f| - 1.
  Return the updated flow.
```

Correctness:

Case 1 : If $f(e^*)$ was unsaturated, reducing capacity doesn't break feasibility, and since no augmenting path existed before, none exists now; f is still maximum.

Case 2: $f(e^*) = c(e^*)$

Decrement flow by 1 along $P_1 \to e^* \to P_2$. The resulting flow f' is feasible in G' since e^* now carries $c(e^*) - 1 = c'(e^*)$, and all other edges are unchanged. We have $|f'| = |f| - 1$.

Now run BFS in the residual graph $G'_{f'}$. We consider two subcases.

If no s-t path exists: By the max-flow min-cut theorem, f' is a maximum flow in G', so $|f^*| = |f| - 1$.

If an s-t path P exists: Augment f' by exactly 1 unit along P, yielding f'' with $|f''| = |f|$. We now show f'' is a maximum flow in G', i.e., $|f^*| = |f|$. Observe that for any s-t cut $(S, \bar{S})$ in G':

$$c'(S, \bar{S}) \geq c(S, \bar{S}) - 1 \geq |f| - 1$$

where the first inequality holds since c' and c differ only on e^* by 1, and the second since every cut in the original graph has capacity at least $|f|$ (by max-flow min-cut on f). Taking the minimum over all cuts:

$$|f^*| = \min_{(S,\bar{S})} c'(S, \bar{S}) \geq |f| - 1$$

But also $|f^*| \leq |f|$ since we only reduced a capacity. So $|f^*| \in \{|f| - 1,\ |f|\}$. Since f'' is a feasible flow in G' with $|f''| = |f|$, we conclude $|f^*| = |f|$ and f'' is a maximum flow.

Running Time:

Each BFS/DFS takes $\mathcal{O}(m+n)$. We perform at most 3 traversals. Total time is $\mathcal{O}(m+n)$.

Rubric: Algorithm [4 marks]

- 1 mark for correctly handling Case 1 (unsaturated edge).
- 3 marks for complete Case 2: finding P_1, P_2, decrementing flow along $P_1 \to e^* \to P_2$, and running BFS in $G'_{f'}$ to check for an augmenting path.
- -2 marks if Steps 4–5 are missing (BFS check and the two subcases on whether a path exists or not).

Correctness [3 marks]

- 1 mark for the unsaturated case (Case 1): arguing that f remains feasible and maximum in G'.
- 2 marks for the saturated case (Case 2): correctly handling both subcases — (i) no augmenting path implies $|f^*| = |f| - 1$, and (ii) augmenting path exists implies $|f^*| = |f|$, with a formal argument (e.g., via min-cut) justifying why a single augmentation of 1 unit suffices.
- -2 marks if the proof invokes results without justification, e.g., writing "as shown in class" in place of a formal argument.

Common Mistakes

1. Decrementing flow by 1 along $P_1 \to e^* \to P_2$ and directly concluding $|f^*| = |f| - 1$, without checking for an augmenting path in $G'_{f'}$.
2. Rewriting the algorithm in prose under the correctness section instead of providing a formal proof of optimality.

Question 4 (8 Marks)**.** The problem of searching for cycles in graphs arises naturally in financial trading applications. Consider a firm that trades shares in n different companies. For each pair $i \neq j$, they maintain a trade ratio r_{ij}, meaning that one share of i trades for r_{ij} shares of j. Here we allow the ratio to be fractional; that is, $r_{ij} = \frac{2}{3}$ means that you can trade three shares of i to get two shares of j. A trading cycle for a sequence of shares $i_1, i_2, ..., i_k$ consists of successively trading shares in company i_1 for shares in company i_2, then shares in company u_2 for shares in company t_3, and so on, finally trading shares in t_k back to shares in company i_1. After such a sequence of trades, one ends up with shares in the same company i_1 that one starts with. Trading around a cycle is usually a bad idea, as one tends to end up with fewer shares than one started with. But occasionally, for short periods of time, there are opportunities to increase shares. We call such a cycle an opportunity cycle, if trading along the cycle increases the number of shares. This happens exactly if the product of the ratios along the cycle is greater than 1. In analyzing the state of the market, a firm engaged in trading would like to know if there are any opportunity cycles. Give a polynomial-time algorithm that finds such an opportunity cycle, if one exists.

[Marks distribution: 1 marks for formulating the problem as cycle detection, 4 marks for the algorithm, 2 marks for correctness, 1 mark for running time analysis.]

Solution:

Formulation:

A trading cycle is an opportunity cycle iff $r_{i_1 i_2} \cdot r_{i_2 i_3} \cdot \cdots \cdot r_{i_k i_1} > 1$. Taking negative logarithms, we define edge weights $w_{ij} = -\log(r_{ij})$. The condition becomes $\sum w_{\text{edges}} < 0$. Thus, finding an opportunity cycle equates to finding a negative-weight cycle.

Algorithm:

1. Build directed graph G with n nodes and edges weighted $w_{ij} = -\log(r_{ij})$.
2. Add a virtual source node s with zero-weight edges to all n nodes.
3. Run Bellman-Ford from s for n iterations.
4. If any edge relaxes in the n-th iteration, trace back parent pointers to recover the negative cycle.

Correctness:

Bellman-Ford correctly detects negative cycles. If a distance still decreases in round n,

the path has n edges and must contain a repeated node, forming a negative cycle. This corresponds directly to a product of ratios > 1.

Running Time:
Bellman-Ford on n nodes and $m \leq n(n-1)$ edges runs in $\mathcal{O}(n \cdot m) = \mathcal{O}(n^3)$ time, which is polynomial.

Rubric:

- Same as marks distribution.
- -2 marks if cycle is not returned.

Question 5 (6 Marks)**.** We have an array A with n integers sorted in increasing order and an array B with n integers sorted in decreasing order. Consider the following game. You can choose any index $i \in \{0, 1, ..., n-1\}$. Then you will be given either $A[i]$ or $B[i]$ points, whichever is smaller. Naturally, you would like to choose an index i that gives you the maximum number of points.

Example: A= [2, 5, 7, 10, 11, 12, 15, 18, 25, 45], B = [50, 40, 32, 25, 22, 20, 18, 16, 10, 5]. Here, the maximum number of points you can get is 16, by choosing $i = 7$ How will you find such an index i in $\mathcal{O}(log\ n)$ time?

[Marks distribution: 2 marks for the correct idea, 2 marks for writing the correct psuedo code and 2 marks for proving the correctness of the algorithm.]

Solution:

Key Insight:
Define $f(i) = A[i] - B[i]$. Since A is non-decreasing and B is non-increasing, f is strictly non-decreasing. The optimal index is located where the two arrays intersect (where $A[i] \approx B[i]$). We binary search for the largest index where $A[i] \leq B[i]$.

Algorithm:

```
BinarySearchOptimal(A, B, n):
  lo = 0, hi = n - 1
  while lo < hi:
    mid = (lo + hi + 1) / 2
    if A[mid] <= B[mid]: lo = mid
    else: hi = mid - 1
  best = lo
  if lo + 1 <= n - 1 and min(A[lo+1], B[lo+1]) > min(A[lo], B[lo]):
    best = lo + 1
  return best
```

Correctness:
Let i^* be the last index with $A[i] \leq B[i]$. For $i < i^*$, the minimum is $A[i]$ (which only increases up to i^*). For $i > i^*+1$, the minimum is $B[i]$ (which strictly decreases). Therefore, the global maximum of $\min(A[i], B[i])$ must occur at either i^* or $i^* + 1$. The binary search finds this in $\mathcal{O}(\log n)$ time.

Rubric: Same as marks distribution.

Question 6 (6 Marks)**.** Consider the makespan minimization problem on three processors. You are given n jobs with processing times $p_1, p_2, ..., p_n$. The goal is to assign each job to one of

the three processors so that the makespan (the maximum total load assigned to any processor) is minimized. Design a pseudo-polynomial-time algorithm to compute the minimum makespan exactly. Your running time should be polynomial in n and $\sum_{i=1}^{n} p_i$. Explain your algorithm and analyze its running time.

[Marks distribution: 3 marks for the algorithm, 1.5 marks for the correctness, 1.5 marks for the running time analysis.]

Solution:

Let $P_j = \sum_{i=1}^{j} p_i$. Create a 3-dimensional Boolean table T with dimensions $n \times (P_n + 1) \times (P_n + 1)$. $T[i, x, y]$ will denote whether it is possible to assign the first i jobs in a way that the total load on processor 1 is x, the total load on machine 2 is y, and the total load on machine 3 is $P_i - x - y$. For any i, x, y, the table entry is computed as follows:

$$T[i, x, y] = T[i-1, x-p_i, y] \vee T[i-1, x, y-p_i] \vee T[i-1, x, y]$$

This indicates that the i-th job will be assigned to one of the three machines, determining the remaining loads. Clearly, the table T can be computed in increasing order of i, x, y.

Base case: $T[1, p_1, 0] = T[1, 0, p_1] = T[1, 0, 0] =$ TRUE. For any other entry, $T[1, \cdot, \cdot] =$ FALSE.

To find the minimum makespan, we iterate over all entries $T[n, x, y]$ of the table which are TRUE. For each entry, the makespan is $\max(x, y, P_n - x - y)$. We find the minimum of all these makespans. Formally:

$$\min\{\max(x, y, P_n - x - y) \mid T[n, x, y] = \text{TRUE}\}$$

The running time of this algorithm is $\mathcal{O}(n \cdot P_n^2)$. Thus, it is pseudo-polynomial.

Rubric:

- Same as marks distribution
- -1 marks if base case is missing or incorrect
- No marks awarded if algorithm is incorrect (greedy, flow formulation etc.)

Question 7 (8 Marks)**.** Suppose you are given a connected graph G, with edge costs that are all positive and distinct.

(a) [4 Marks] Prove that G has a unique minimum spanning tree.

(b) [2 Marks] Let T be a minimum spanning tree for this instance. Now suppose we replace each edge cost c_e by its square, c_e^2, thereby creating a new instance of the problem with the same graph but different costs. Is it true or false that T must still be a minimum spanning tree for this new instance? Justify your answer.

(c) [2 Marks] Suppose we are given an instance of the Shortest $s - t$ Path problem on a directed graph G. We assume that all edge costs are positive and distinct. Let P be a minimum-cost $s - t$ path for this instance. Now suppose we replace each edge cost c_e by its square, c_e^2, thereby creating a new instance of the problem with the same graph but different costs. Is it true or false? P must still be a minimum-cost $s - t$ path for this new instance. Justify your answer.

Solution:

(a) Proof of Uniqueness:
Suppose T and T' are distinct MSTs. Let $e = (u, v)$ be the minimum-cost edge in their symmetric difference $T \triangle T'$. W.l.o.g, let $e \in T \setminus T'$. Adding e to T' forms a cycle C. Every other edge $f \in C \setminus \{e\}$ must be in T as well (if $f \notin T$, then $f \in T \triangle T'$, and $c(f) > c(e)$, but swapping e into T' for f would lower the weight of T', contradicting T' being an MST). But if all $f \in C \setminus \{e\}$ are in T, then $T \cup \{e\}$ contains a cycle in T itself, which contradicts T being a tree. Hence, $T = T'$.

Assessment Rubric: 2 marks for correct construction of logical reasoning. 2 marks for proper formalism in accordance with standard proof-writing conventions and for correct logical steps.

(b) TRUE. The original edge costs are strictly positive and distinct. When these costs are squared, the resulting values remain positive and distinct as any positive square has two real roots (one positive and one negative of equal magnitude), the restriction to strictly positive original costs guarantees that no two distinct edge weights can yield the same squared value. Also, the squaring operation is strictly monotonically increasing for all $x > 0$, perfectly preserving the relative ordering of all edge weights such that $c(e) < c(f) \iff c(e)^2 < c(f)^2$.

Assessment Rubric: 1 mark for correct answer. 1 mark for correct reasoning.

(c) FALSE.

Justification: Unlike the Minimum Spanning Tree problem (which depends only on the relative ordering of individual edge weights), the Shortest Path problem depends on the *sum* of the edge weights along a path. The function $f(x) = x^2$ penalizes individual edges with large weights much more heavily than a sequence of multiple edges with smaller weights.

Consider a counterexample graph with two possible paths from s to t:

- **Path** P consists of a single edge with cost $c = 6$. The total cost of P is 6.
- **Path** P' consists of two edges with costs $c_1 = 3$ and $c_2 = 4$. The total cost of P' is $3 + 4 = 7$.

In the original graph, P is the minimum-cost $s - t$ path because $6 < 7$.
After we replace each edge cost with its square:

- The new cost of **Path** P is $6^2 = 36$.
- The new cost of **Path** P' is $3^2 + 4^2 = 9 + 16 = 25$.

In the new instance, the cost of P' (25) is strictly less than the cost of P (36). Therefore, P is no longer the minimum-cost $s - t$ path.

Assessment Rubric: 1 mark for correct answer. 1 mark for correct counter example and/or correct reasoning.

Question 8 (5 Marks)**.** Consider the following maximization version of the 3-Dimensional Matching problem. Given disjoint sets X, Y, and Z, and a set $T \subseteq X \times Y \times Z$ of ordered triples, a subset $M \subseteq T$ is a 3-dimensional matching if each element of $X \cup Y \cup Z$ is contained in at most one of these triples. The Maximum 3-Dimensional Matching problem is to find a 3-dimensional matching M of maximum size. The size of the matching is the number of triples it contains. You may assume $|X| = |Y| = |Z|$, if you wish. Give a polynomial-time algorithm that finds a 3-dimensional matching of size at least $\frac{1}{3}$ times the maximum possible size.

Solution:

Algorithm — Greedy:

```
M = empty set
while T is non-empty:
  Pick any triple t = (x, y, z) in T
  Add t to M
  Remove from T all triples sharing an element with t
return M
```

Approximation Ratio:

M is a valid matching. When we add $t = (x, y, z)$ to M, we remove at most 3 triples from the optimal matching OPT (one containing x, one containing y, and one containing z). Each step blocks at most 3 triples from OPT. If $|M| = k$, we block at most $3k$ triples, meaning $|OPT| \leq 3k = 3|M|$. Therefore, $|M| \geq \frac{1}{3}|OPT|$.

2 marks for correct algorithm. 1 mark for analysis of time complexity. 2 marks for proof of approximation guarantee.